DISEÑO DE COMPILADORES



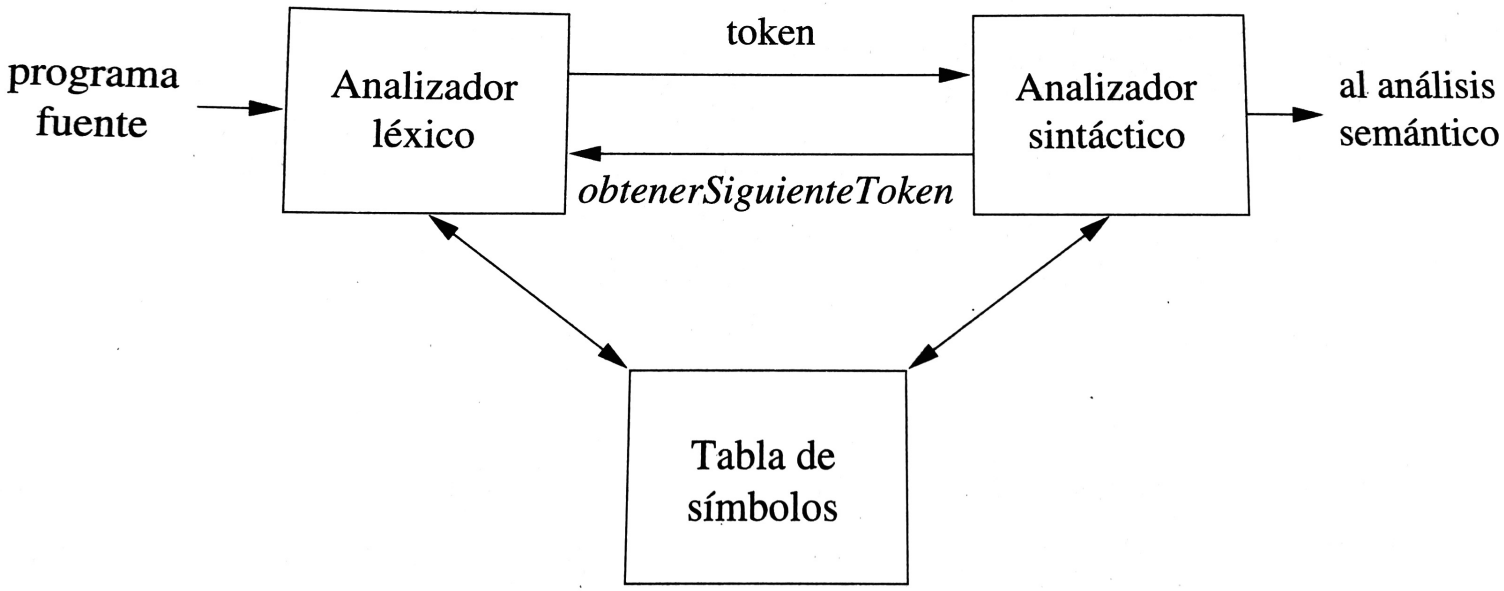
Capítulo 3

Análisis Léxico

Observación. *El presente documento sirve de complemento al contenido que referencia y no se constituye en el detalle del mismo, ya que conceptos y contenidos pueden haber sido desarrollados en forma presencial, virtual o asignado como ejercicio. Ante cualquier duda se recomienda consultar con el profesor a cargo de la cátedra.*

Profesor: Sergio A. Aranda

ANÁLISIS LÉXICO
Introducción



LEXEMAS, TOKENS Y PATRONES
Definición

- **Lexema**: es una secuencia de caracteres del programa fuente, que coincide con un patrón. Es una instancia de un token.

- **Token**: es un nombre que define una clase o tipo de patrón. Puede ser identificado por el nombre o como un par ordenado compuesto del nombre del token y del valor del mismo.

- **Patrón**: es una descripción, generalmente en lenguaje natural, de que lexemas pertenecen a un token. Es la definición del token. Puede utilizarse además esquemas de definición por extensión o comprensión.

TOKEN	PATRÓN	LEXEMA
número	Cualquier constante numérica	3,14159 | 0 | 25 | …
Comparación	< o > o <= o >= o == o !=	==, !=
ELSE	La letra e seguida de la l, la s y la e.	else

ALFABETOS, CADENAS Y LENGUAJES
Definición

- **Alfabeto**: conjunto finito de símbolos. Ejemplos: {0,1}, letras, dígitos, el código ASCII.

- Una **cadena** sobre un *alfabeto* : secuencia finita de símbolos obtenidos a partir de un alfabeto. La longitud de una cadena está definida por la cantidad de símbolos que la forman. La cadena “universidad” tiene una longitud igual a 11.

  La cadena vacía, representada por ε, tiene una longitud igual a cero.

- Un **lenguaje** es cualquier conjunto contable de *cadenas* definidas sobre un *alfabeto*. Pueden ser definidas por extensión o comprensión.

LENGUAJES
Operación sobre lenguajes

OPERACIÓN	DEFINICIÓN Y NOTACIÓN
Unión de L y M	$L \cup M = \{ s \mid s \text{ está en } L \text{ o } s \text{ está en } M \}$
Concatenación de L y M	$LM = \{st \mid s \text{ está en } L \text{ y } t \text{ está en } M \}$
Cerradura de Kleene de L	$L^* = \bigcup_{i=0}^{\infty} L^i$
Cerradura positivo de L	$L^+ = \bigcup_{i=1}^{\infty} L^i$

Sea L el conjunto de letras = {a, b, c, …, z} y D el conjunto de digitos = {0, 1, …, 9}, entonces:

- L U D = {a, b, …, 0, 1, 2, …}
- L D = {a0, a1, a2, …, c8, c9, d0, …}
- L^4 = {aaaa, aaab, aaac, …, azzz, bbbb, …}
- D^+
- L (L U D)

EXPRESIONES REGULARES
Definición

BASE: Hay dos reglas que forman la base:

1. ϵ es una expresión regular, y $L(\epsilon)$ es $\{\epsilon\}$; es decir, el lenguaje cuyo único miembro es la cadena vacía.
2. Si a es un símbolo en Σ, entonces $\mathbf{a}$ es una expresión regular, y $L(\mathbf{a}) = \{a\}$, es decir, el lenguaje con una cadena, de longitud uno, con a en su única posición. Tenga en cuenta que por convención usamos cursiva para los símbolos, y negrita para su correspondiente expresión regular.[1]

INDUCCIÓN: Hay cuatro partes que constituyen la inducción, mediante la cual las expresiones regulares más grandes se construyen a partir de las más pequeñas. Suponga que r y s son expresiones regulares que denotan a los lenguajes $L(r)$ y $L(s)$, respectivamente.

1. $(r)|(s)$ es una expresión regular que denota el lenguaje $L(r) \cup L(s)$.
2. $(r)(s)$ es una expresión regular que denota el lenguaje $L(r)L(s)$.
3. $(r)^*$ es una expresión regular que denota a $(L(r))^*$.
4. (r) es una expresión regular que denota a $L(r)$. Esta última regla dice que podemos agregar pares adicionales de paréntesis alrededor de las expresiones, sin cambiar el lenguaje que denotan.

EXPRESIONES REGULARES
Leyes algebraicas

LEY	DESCRIPCIÓN
$r\|s = s\|r$	$\|$ es conmutativo
$r\|(s\|t) = (r\|s)\|t$	$\|$ es asociativo
$r(st) = (rs)t$	La concatenación es asociativa
$r(s\|t) = rs\|rt;\ (s\|t)r = sr\|tr$	La concatenación se distribuye sobre
$\epsilon r = r\epsilon = r$	ϵ es la identidad para la concatenación
$r^* = (r\|\epsilon)^*$	ϵ se garantiza en un cerradura
$r^{**} = r^*$	* es idempotente

Leyes algebraicas que se aplican a las expresiones regulares

EXPRESIONES REGULARES
Extensiones de las expresiones regulares

Las siguientes son extensiones utilizadas en las expresiones regulares como consecuencia de su definición y leyes algebraicas:

- Una o más instancias. Cerradura positiva de Kleene. Representado con el operador +. Así, $r^+ = r\, r^*$, y $r^* = (r^+ \mid \varepsilon)$.
- Cero o una instancia. Representado con el operador ?. Asi es posible establecer que $r? = r \mid \varepsilon$.
- Clase de caracteres. Es una abreviación para representar una secuencia de símbolos. Ejemplo: (a|b|c ..|z) = [a-z]

EXPRESIONES REGULARES
Ejemplos

1. Todas las cadenas de letras minúsculas que comiencen y terminen con la letra 'a'.

2. Cadenas de a's y b's que no tengan tres b's consecutivas.

3. Cadenas de a's y b's que terminan en 'a', 'aa', o 'ab'.

4. Cadenas de a's y b's que no poseen un número impar de a's consecutivas seguidas de un número impar de b's consecutivas.

EXPRESIONES REGULARES
Ejemplos

1. Todas las cadenas de letras minúsculas que comiencen y terminen con la letra 'a'.

   a ([a-z]* a)?

2. Cadenas de a's y b's que no tengan tres b's consecutivas.

   a* b? (ab | ba)* b? a*

3. Cadenas de a's y b's que terminan en 'a', 'aa', o 'ab'.

   (a | b)* a (a | b | ε)

4. Cadenas de a's y b's que no poseen un número impar de a's consecutivas seguidas de un número impar de b's consecutivas.

   (aa | b)* (a | bb)*

DEFINICIONES REGULARES
Ejemplos

instr	→	**if** *expr* **then** *instr*
	|	**if** *expr* **then** *instr* **else** *instr*
	|	ϵ
expr	→	*term* **oprel** *term*
	|	*term*
term	→	**id**
	|	**numero**

BNF (analizador sintáctico)

digito	→	[0-9]
digitos	→	*digito*$^+$
numero	→	*digitos* (. *digitos*)? (E [+-]? *digitos*)?
letra	→	[A-Za-z]
id	→	*letra* (*letra* | *digito*)*
if	→	if
then	→	then
else	→	else
oprel	→	< | > | <= | >= | = | <>

Def. Regular (analizador léxico)

DIAGRAMA DE TRANSICIÓN DE ESTADOS
Ejemplos

oprel → < | > | <= | >= | = | <>

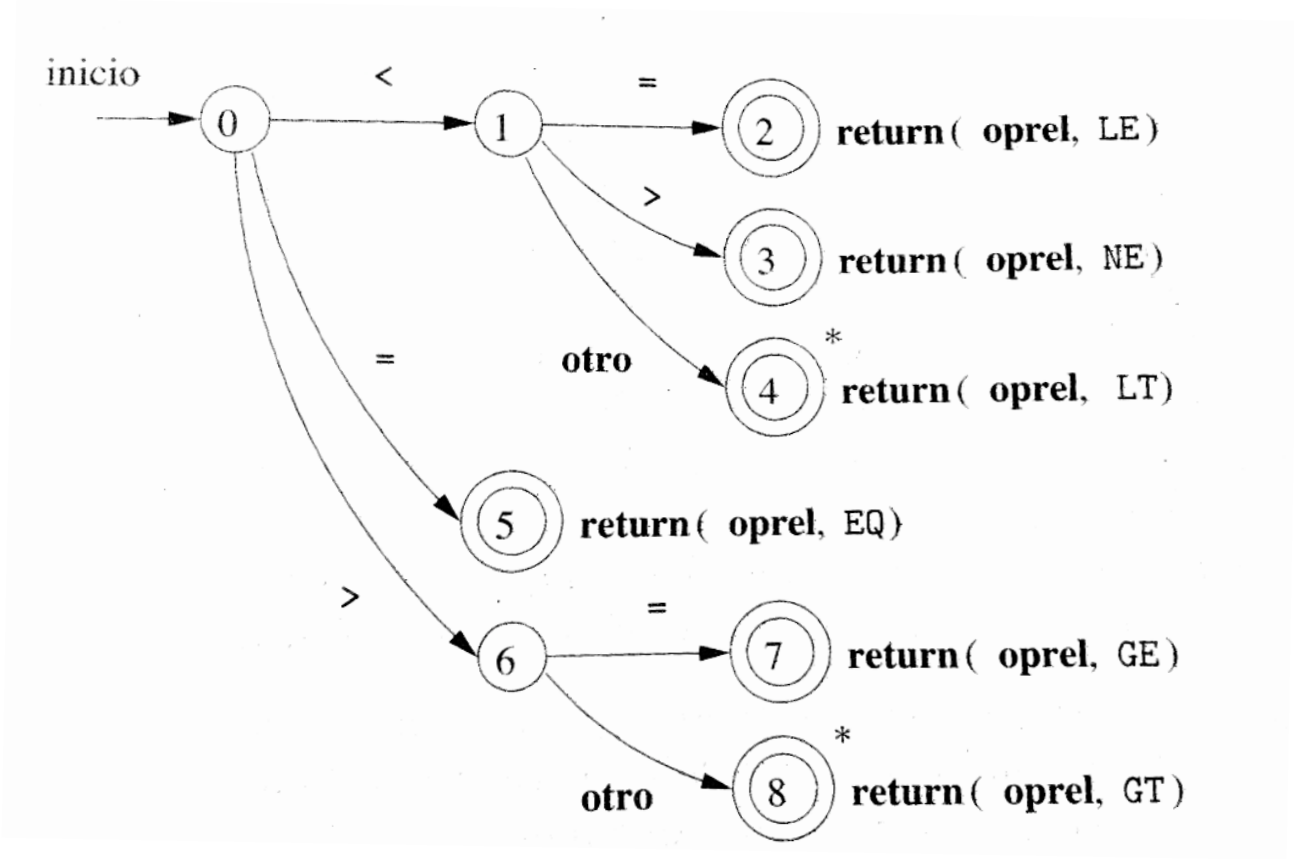


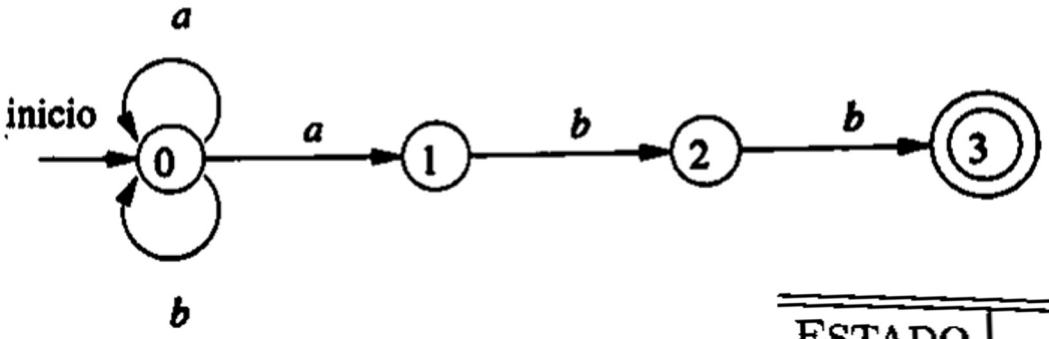
DIAGRAMA DE TRANSICIÓN DE ESTADOS
Ejemplos

```
TOKEN obtenerOpRel()
{
    TOKEN tokenRet = new (OPREL);
    while(1) { /* repite el procesamiento de caracteres hasta que
                  ocurre un retorno o un fallo */
        switch(estado) {
            case 0: c = sigCar();
                    if ( c == '<' ) estado = 1;
                    else if ( c == '=' ) estado = 5;
                    else if ( c == '>' ) estado = 6;
                    else fallo(); /* el lexema no es un oprel */
                    break;
            case 1: ...
            ...
            case 8: retractar();
                    tokenRet.atributo = GT;
                    return(tokenRet);
        }
    }
```

Un *autómata finito no determinista* (AFN) consiste en:

1. Un conjunto finito de estados S.
2. Un conjunto de símbolos de entrada Σ, el *alfabeto de entrada*. Suponemos que ϵ, que representa a la cadena vacía, nunca será miembro de Σ.
3. Una *función de transición* que proporciona, para cada estado y para cada símbolo en $\Sigma \cup \{\epsilon\}$, un conjunto de *estados siguientes*.
4. Un estado s_0 de S, que se distingue como el *estado inicial*.
5. Un conjunto de estados F, un subconjunto de S, que se distinguen como los *estados aceptantes* (o *estados finales*).

AFN que reconoce el lenguaje para la expresión regular (a|b)* abb

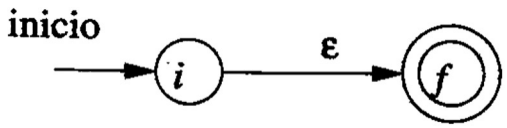


ESTADO	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

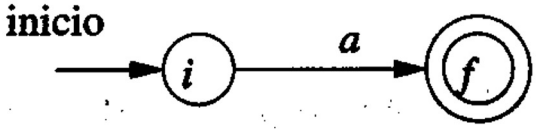
AUTÓMATAS FINITOS
Algoritmo de Thompson

Construcciones de Thompson que permite obtener un AFN a partir de una expresión regular

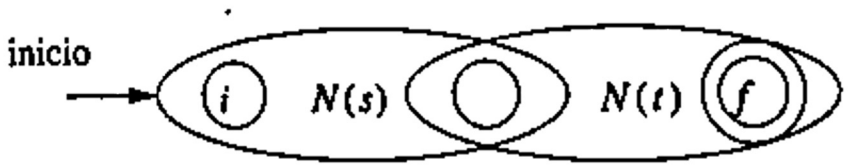
ε



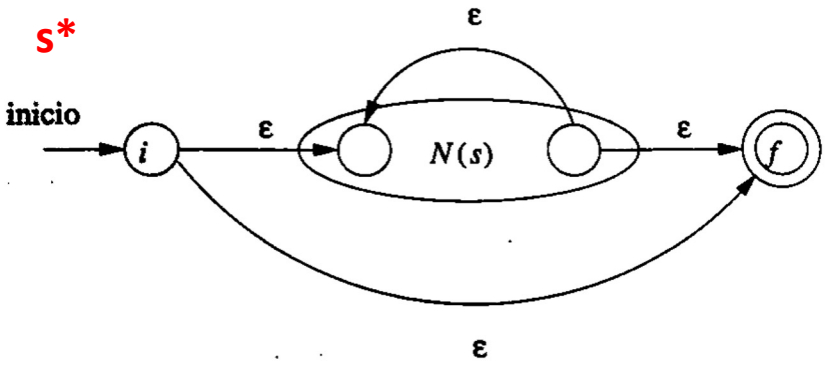
a



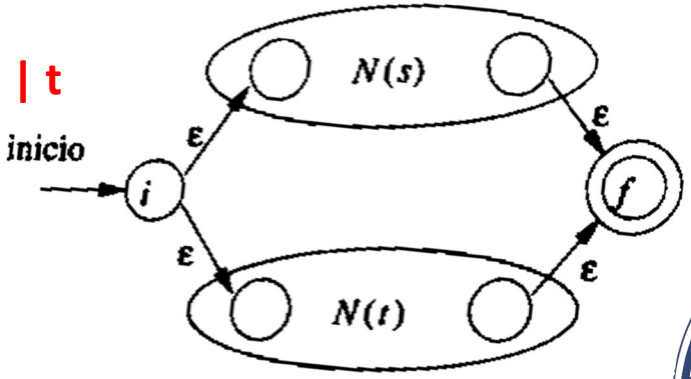
s t



s*

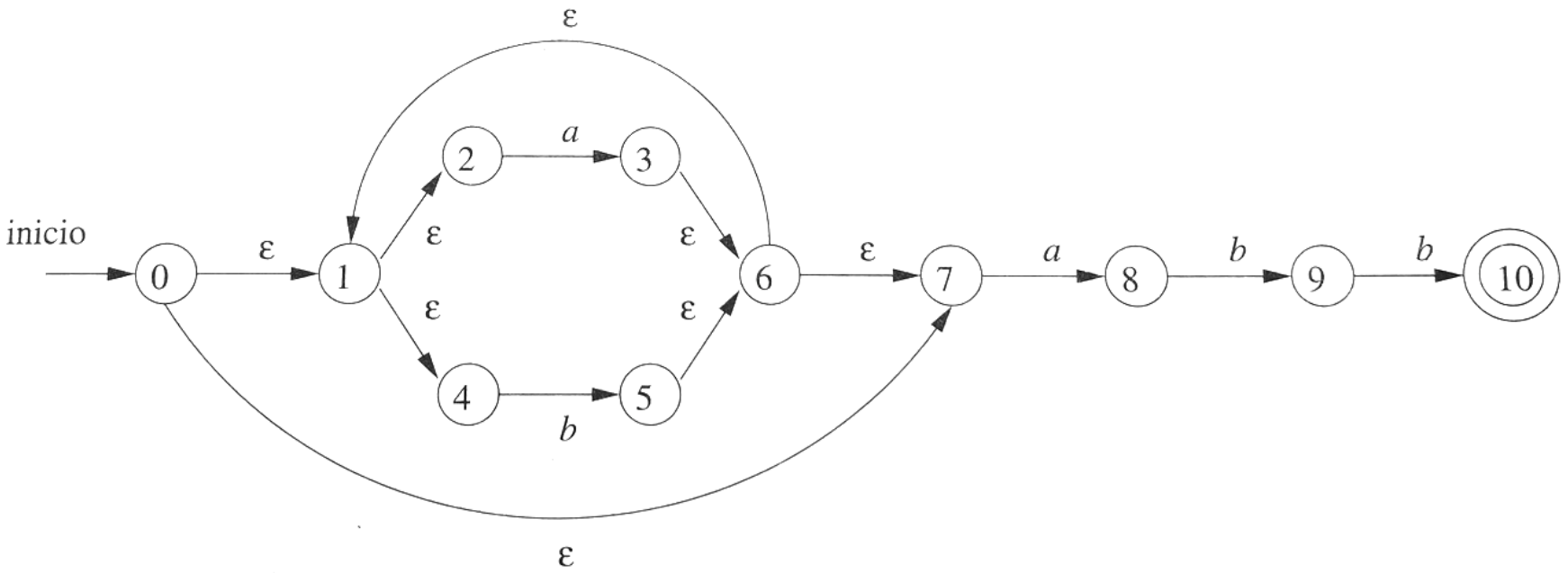


s | t



AUTÓMATAS FINITOS
Algoritmo de Thompson

(a|b)* abb



AUTÓMATAS FINITOS DETERMINÍSTAS
Definición

Un autómata finito deterministico (AFD), es un autómata finito no deterministico, pero con las siguientes características:

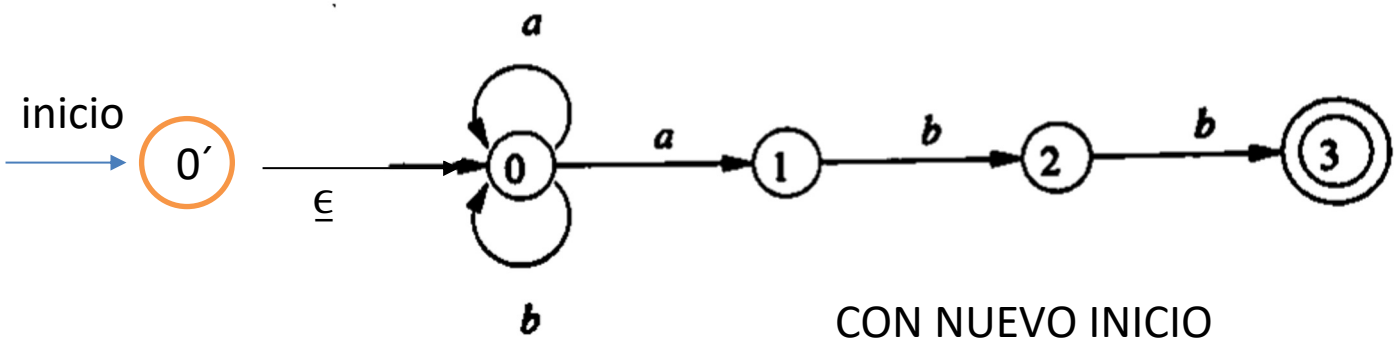
a. No tiene transiciones definidas para el símbolo vacío
b. Dado un estado i, existe sólo una transición que sale de i a un estado siguiente para un símbolo del alfabeto a.
c. Para un estado i dado, existen n transiciones posibles para n símbolos diferentes del alfabeto.

CONVERSIÓN DE UN AFN A UN AFD
Algoritmo de Subconjuntos

Operaciones sobre los estados del AFN

OPERACIÓN	DESCRIPCIÓN
ϵ-*cerradura*(s)	Conjunto de estados del AFN a los que se puede llegar desde el estado s del AFN, sólo en las transiciones ϵ.
ϵ-*cerradura*(T)	Conjunto de estados del AFN a los que se puede llegar desde cierto estado s del AFN en el conjunto T, sólo en las transiciones ϵ; $= \cup_{s \text{ en } T}$ ϵ-*cerradura*(s).
mover(T, a)	Conjunto de estados del AFN para los cuales hay una transición sobre el símbolo de entrada a, a partir de cierto estado s en T.

AUTÓMATAS FINITOS
Autómatas Finitos No Determinísticos



ORIGINAL
Destados={ {0}, {0, 1}, {0, 2}

c-e(mover({0}, a)) = c-e({0, 1})={0, 1}
c-e(mover({0}, b)) = c-e({0}} = {0}

c-e(mover({0, 1}, a)) = c-e({0, 1})={0, 1}
c-e(mover({0, 1}, b)) = c-e({0, 2}} = {0, 2}

CON NUEVO INICIO
Destados={ {0´, 0}, {0, 1}, {0}, {0, 2}

c-e(mover({0´, 0}, a)) = c-e({0, 1})={0, 1}
c-e(mover({0´, 0}, b)) = c-e({0}} = {0}

c-e(mover({0, 1}, a)) = c-e({0, 1})={0, 1}
c-e(mover({0, 1}, b)) = c-e({0, 2}} = {0, 2}

CONVERSION DE UN AFN A UN AFD
Algoritmo de subconjuntos

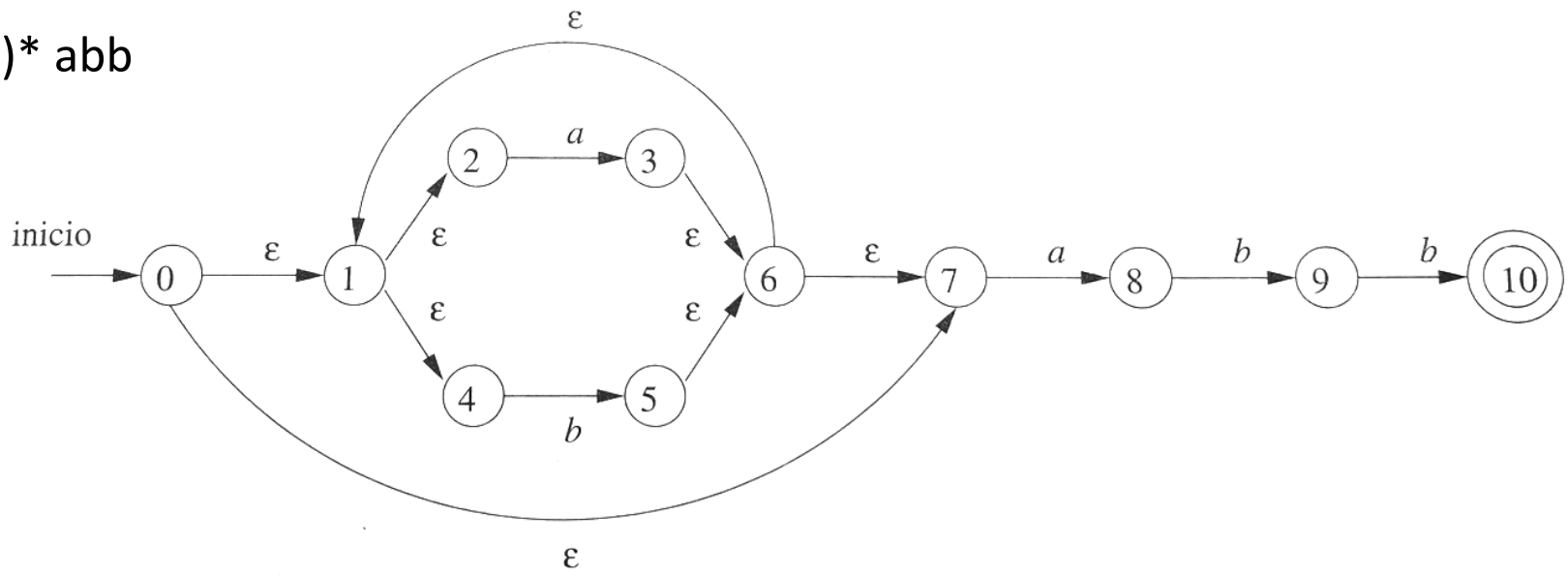
Algoritmo de conversión de un AFN a un AFD

Inicializar Destados con ε-cerradura(s_0),

```
while ( hay un estado sin marcar T en Destados ) {
        marcar T;
        for ( cada símbolo de entrada a ) {
                U = ε-cerradura(mover(T, a));
                if ( U no está en Destados )
                        agregar U como estado sin marcar a Destados;
                Dtran[T, a] = U;
        }
}
```

CONVERSION DE UN AFN A UN AFD
Ejemplo

(a|b)* abb



CONVERSION DE UN AFN A UN AFD
Minimización de un AFD

MÉTODO:

1. Empezar con una partición inicial Π con dos grupos, F y $S - F$, los estados de aceptación y de no aceptación de D.
2. Aplicar el procedimiento de la figura 3.64 para construir una nueva partición Π_{nueva}.

```
al principio, dejar que Π_nueva = Π;
for (cada grupo G de Π ) {
        particionar G en subgrupos, de forma que dos estados s y t
                se encuentren en el mismo subgrupo, si y sólo si para todos
                los símbolos de entrada a, los estados s y t tienen transiciones
                sobre a hacia estados en el mismo grupo de Π;
        /* en el peor caso, un estado estará en un subgrupo por sí solo */
        sustituir G en Π_nueva por el conjunto de todos los subgrupos formados;
}
```

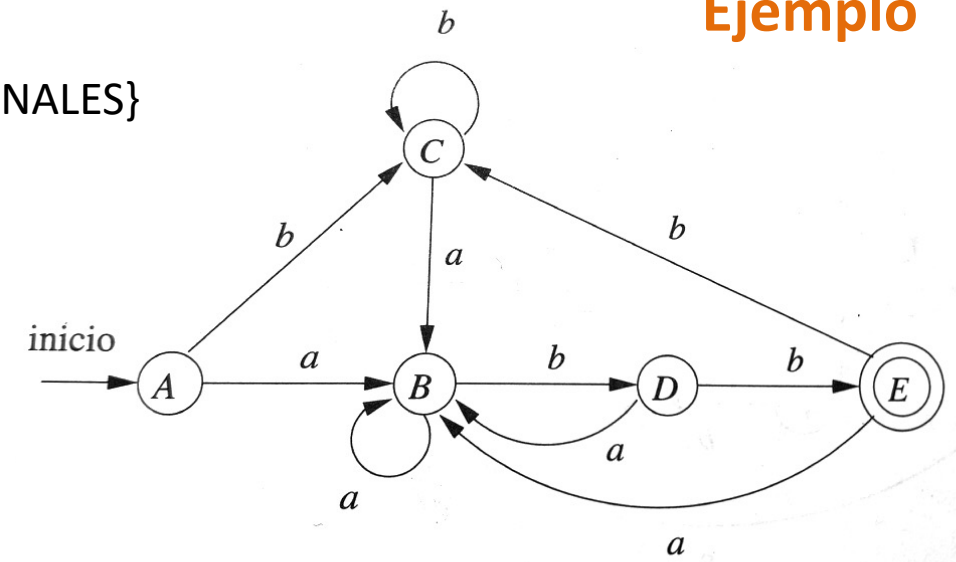
ALGORITMO DE MINIMIZACIÓN
Ejemplo

1. PI={ ESTADOS_FINALES, ESTADOS_NO_FINALES}
ESTADOS_FINALES={ E }
ESTADOS_NO_FINALES={ A, B, C, D }
PI={ $\{E\}_I$, $\{A, B, C, D\}_{II}$ }

2. PInueva ={ $\{E\}_I$, $\{A, B, C\}_{II}$, $\{D\}_{III}$ }

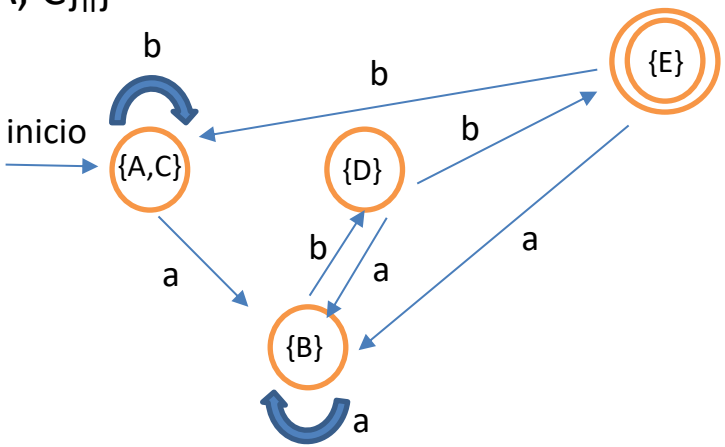
$PInueva_2$= {$\{E\}_I$, $\{D\}_{III}$, $\{A, C\}_{II}$, $\{B\}_{IV}$ }

$PInueva_3$= { $\{E\}_I$, $\{D\}_{III}$, $\{B\}_{IV}$, $\{A, C\}_{II}$}



Estado	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

Estado	a	b
{A,C}	{B}	{A, C}
{B}	{B}	{D}
{D}	{B}	{E}
{E}	{B}	{A,C}



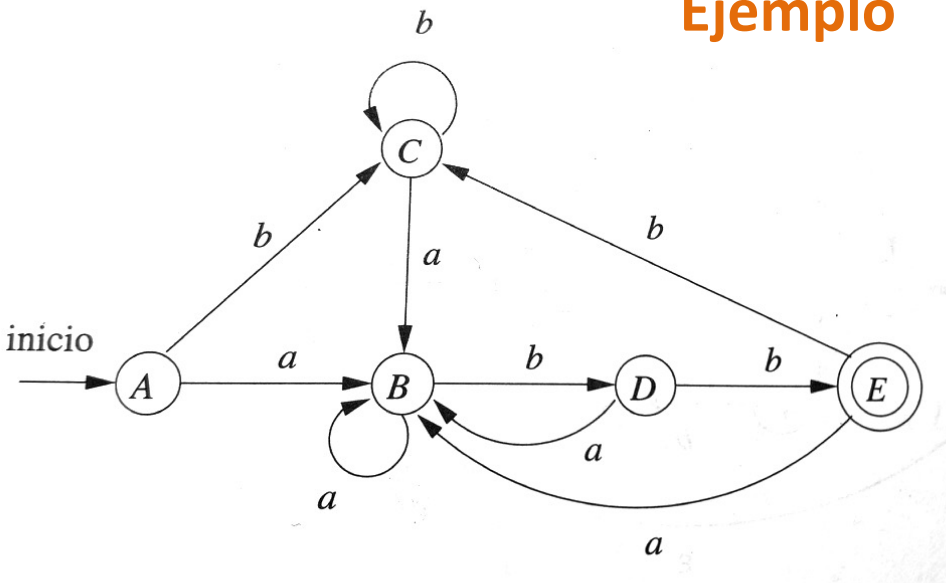
ALGORITMO DE MINIMIZACIÓN
Ejemplo

(a|b)*

(a*|b*)*

Son iguales? Equivalentes?

a. Formulacion matematica
b. Si tienen el mismo AFD minimo



Estado	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

SIMULACIÓN DE UN AFD
Estrategia 2

```
s = s0;
c = sigCar();
while ( c != eof ) {
        s = mover(s, c);
        c = sigCar();
}
if ( s está en F ) return "si";
else return "no";
```

Figura 3.27: Simulación de un AFD

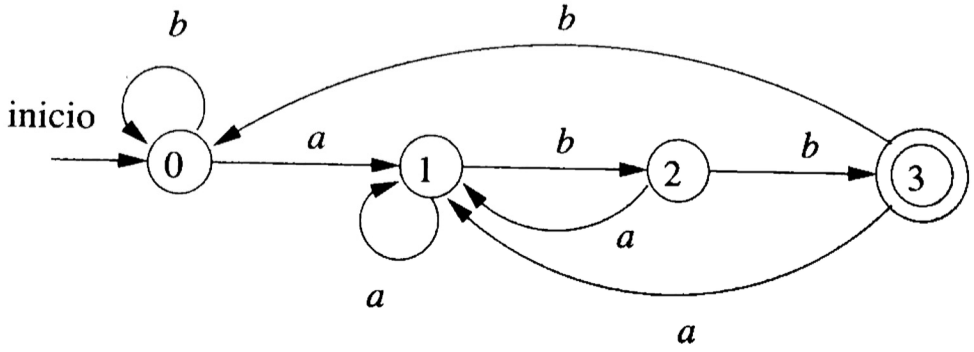


Figura 3.28: AFD que acepta a **(a|b)*abb**

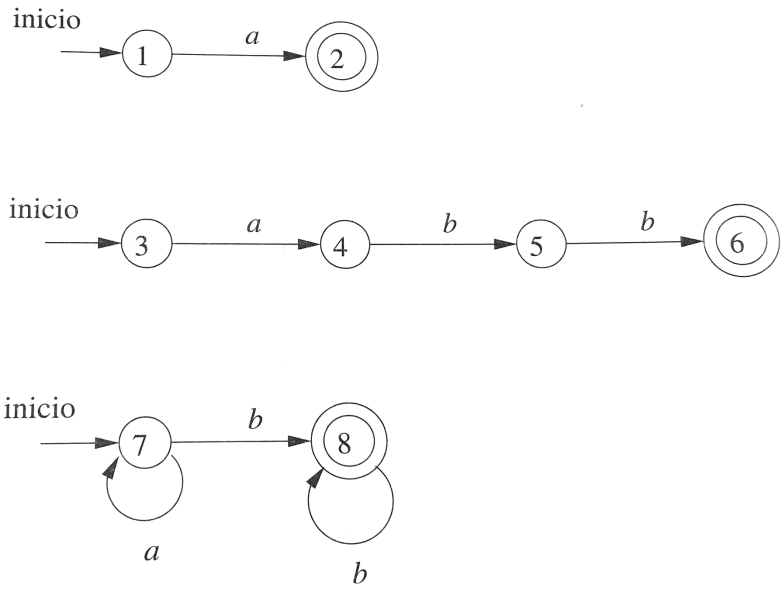
SIMULACIÓN DE UN DIAGRAMA DE FLUJO

Estrategia 1

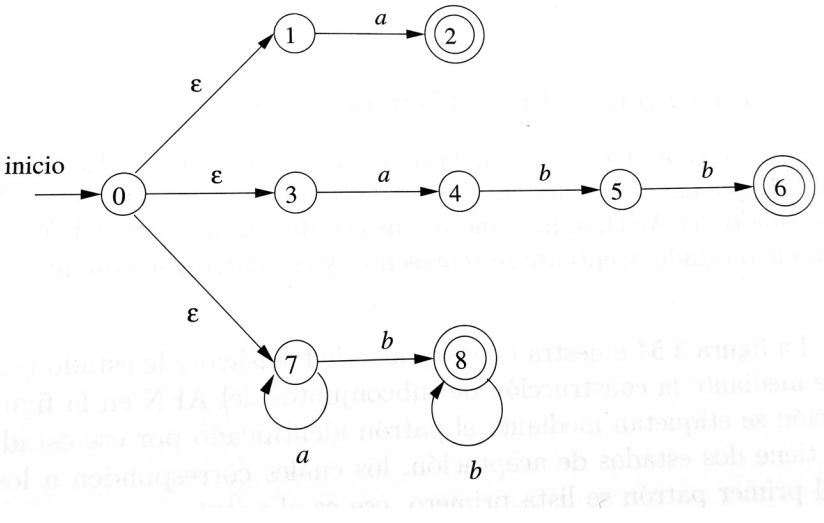
```
complex sigte_complex()
{     while(1) {
          switch (estado) {
          case 0:     c = sigtecar();
               /* c es el carácter de preanálisis */
               if (c==blanco || c==tab || c==línea_nueva) {
                    estado = 0;
                    inicio_lexema++;
                         /* se avanza el inicio del lexema */
               }
               else if (c == '<') estado = 1;
               else if (c == '=') estado = 5;
               else if (c == '>') estado = 6;
               else estado = fallo();
               break;
               .../* aquí van los casos del 1 al 8 */
          case 9:     c = sigtecar();
               if (isletter(c)) estado = 10;
               else estado = fallo();
               break;
          case 10:    c = sigtecar();
               if (isletter(c)) estado = 10;
               else if (isdigit(c)) estado = 10;
               else estado = 11;
               break;
          case 11:    regresa(1); instala_id;
               return ( obtén_complex() );

               .../* aquí van los casos del 12 al 24 */
          case 25:    c = sigtecar();
               if (isdigit(c)) estado = 26;
               else estado = fallo();
               break;
          case 26:    c = sigtecar();
               if (isdigit(c)) estado = 26;
               else estado = 27;
               break;
          case 27:    regresa(1); instala_núm();
               return( NUM );
          }
     }
```

DEFINICIÓN REGULAR
Obteniendo el AFD a partir del AFN

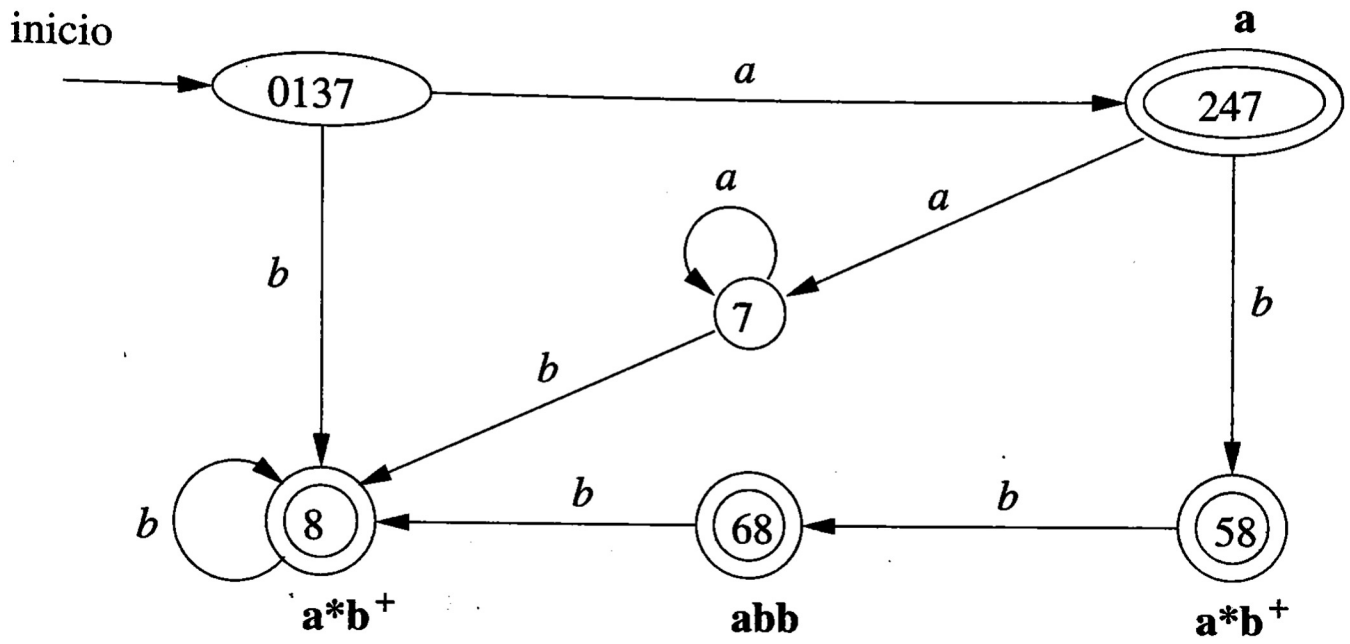


*AFNs para las expresiones regulares a, abb y a*b+*



*AFN único para las expresiones regulares a, abb y a*b+*

DEFINICIÓN REGULAR
Construyendo el AFD



*AFD para las expresiones regulares a, abb y a*b+*